Mackenzie

MBA Engenharia de Dados

Prof. Filipe Quintieri Lima

Guia de Referência SQL

1. Introdução à SQL

A Structured Query Language (SQL) é uma linguagem declarativa fundamental para a gestão e manipulação de dados em sistemas de gerenciamento de banco de dados relacionais (SGBDRs). Desde sua concepção, a SQL tem sido a espinha dorsal para interações com dados, permitindo a consulta, inserção, atualização e definição da estrutura de dados de maneira padronizada. Sua evolução acompanhou o crescimento exponencial do volume e da complexidade dos dados, tornando-se indispensável em ecossistemas de Big Data, Data Warehouses e Business Intelligence (BI), especialmente relevantes para profissionais de Engenharia de Dados.

Breve Histórico

- **1970:** O conceito de banco de dados relacional é proposto por Edgar F. Codd, da IBM. Com base nesse trabalho, Donald D. Chamberlin e Raymond F. Boyce desenvolveram a linguagem SEQUEL (Structured English Query Language), que mais tarde foi abreviada para SQL.
- **1980:** SGBDs comerciais como Oracle, Sybase e DB2 da IBM começam a surgir, implementando a SQL como sua interface principal. Em 1986, a linguagem é padronizada pela ANSI (American National Standards Institute), seguida pela ISO (International Organization for Standardization) em 1987.
- **1990:** Os padrões SQL evoluem, introduzindo funcionalidades avançadas como junções externas (OUTER JOIN), subconsultas e maior rigor sintático. SGBDs de código aberto, como PostgreSQL e MySQL, ganham força.
- **Século 21:** A SQL se consolida como a linguagem universal para dados. Novas versões do padrão incluem suporte a tipos de dados complexos, funções de janela (window functions), expressões de tabela comuns (CTEs), e integração nativa com formatos como XML e JSON. A linguagem se torna central em ecossistemas de Big Data, Data

Warehouses e Business Intelligence (BI).

2. Categorias de Comandos SQL

Os comandos SQL são agrupados em subconjuntos, de acordo com sua funcionalidade principal. As categorias são:

- **DDL (Data Definition Language):** São os comandos da linguagem SQL responsáveis por definir e gerenciar a estrutura dos objetos do banco de dados, incluindo banco de dados, tabelas, índices e visões. Essencial para a organização e manipulação de dados.
- **DML (Data Manipulation Language):** Os comandos DML, são a categoria de comandos dentro da linguagem SQL utilizada para gerenciar e manipular os dados nos bancos relacionais. Ela permite que os usuários interajam com os dados de diversas maneiras.
- **DCL (Data Control Language):** Esta categoria de comandos SQL é utilizada para gerenciar permissões de acesso e controle sobre os dados dentro de um banco de dados.
- **TCL (Transaction Control Language):** As transações são sequências de operações que são executadas como uma única unidade lógica de trabalho. Elas são cruciais para manter a integridade e a consistência dos dados em um banco de dados.

Este guia utiliza a sintaxe e exemplos de comando SQL baseados no PostgreSQL. Para informações adicionais sobre este comando, consulte a documentação oficial do PostgreSQL em https://www.postgresql.org/docs/.

2.1. DDL (Data Definition Language)

A Linguagem de Definição de Dados (DDL) é um subconjunto da SQL (Structured Query Language) que engloba os comandos utilizados para criar, alterar e excluir as estruturas do banco de dados. Em outras palavras, a DDL lida com a arquitetura e o esquema do banco de dados, em vez dos dados.

A DDL é fundamental para a fase de design e manutenção de um banco de dados, permitindo que os desenvolvedores e administradores de banco de dados definam e ajustem a estrutura subjacente onde os dados serão armazenados e gerenciados. Cada comando DDL é executado como uma transação autônoma, ou seja, as alterações são imediatamente confirmadas no banco de dados.

CREATE DATABASE

Cria um novo banco de dados no SGBD. Geralmente, requer privilégios de superusuário ou ter privilégio de CREATEDB.

- **Sintaxe:**

```SQL
CREATE DATABASE nome_do_banco;
```

- **Exemplo:**

```SQL
-- Cria um novo banco de dados para uma aplicação de e-commerce.
CREATE DATABASE loja;
```

ALTER DATABASE

Modifica as propriedades de um banco de dados existente. Uma das operações mais comuns é renomeá-lo.

- **Sintaxe:**

```SQL
ALTER DATABASE nome_do_banco  RENAME TO novo_nome;
```

- **Exemplo:**

```SQL
-- Renomeia o banco de dados.
ALTER DATABASE loja RENAME TO db_loja;
```

DROP DATABASE

Remove permanentemente um banco de dados inteiro, incluindo todas as suas tabelas, dados e objetos em geral.

- **Sintaxe:**

```sql
DROP DATABASE [ IF EXISTS ] nome_do_banco [ (FORCE) ];
```

- **Exemplo:**

```sql
-- Exclui o banco de dados. Esta ação é irreversível.
DROP DATABASE db_loja;
```

CREATE TABLE

O comando CREATE TABLE é utilizado para criar uma nova tabela no banco de dados. Ele permite definir as colunas da tabela, seus respectivos tipos de dados e quaisquer restrições (constraints) que devem ser aplicadas.

- **Sintaxe:**

```sql
CREATE TABLE <nome_da_tabela> (
   <nome_coluna1> <tipo_dados1> [CONSTRAINT1] [CONSTRAINT2]...,
   <nome_coluna2> <tipo_dados2> [CONSTRAINT1] [CONSTRAINT2]...,
   ...
   [CONSTRAINT <nome_restricao> PRIMARY KEY (<coluna_chave_primaria>)],
   [CONSTRAINT <nome_restricao> FOREIGN KEY (<coluna_chave_estrangeira>) REFERENCES
<tabela_referenciada> (<coluna_referenciada>)]
);
```

- **Exemplo:**

```sql
CREATE TABLE pessoas.usuarios (
   id SERIAL PRIMARY KEY,
   nome VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   ativo BOOLEAN DEFAULT TRUE,
   data_cadastro TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP);
```

Principais Constraints:

- PRIMARY KEY: Identificador único para cada registro.
- SERIAL: Tipo especial do PostgreSQL para autoincremento.
- NOT NULL: Garante que a coluna não pode ter valores nulos.
- UNIQUE: Garante que todos os valores na coluna são únicos.
- DEFAULT: Define um valor padrão para a coluna.
- FOREIGN KEY: Cria um vínculo entre tabelas (integridade referencial).

ALTER TABLE

O comando ALTER TABLE é uma declaração SQL fundamental utilizada para modificar a estrutura de uma tabela existente em um banco de dados.

- **Sintaxe**:

```sql
ALTER TABLE <nome_da_tabela>
[ ADD COLUMN <nome_coluna> <tipo_dados> [RESTRIÇÕES...] ]
[ DROP COLUMN <nome_coluna> [CASCADE | RESTRICT] ]
[ ALTER COLUMN <nome_coluna> SET DEFAULT  ]
[ ALTER COLUMN <nome_coluna> DROP DEFAULT ]
[ ALTER COLUMN <nome_coluna> SET NOT NULL ]
[ ALTER COLUMN <nome_coluna> DROP NOT NULL ]
[ ALTER COLUMN <nome_coluna> TYPE <novo_tipo> ]
[ RENAME COLUMN <nome_antigo> TO <novo_nome> ]
[ ADD CONSTRAINT <nome_restricao> <tipo_restricao> ]
[ DROP CONSTRAINT <nome_restricao> [CASCADE | RESTRICT] ]
[ RENAME TO <novo_nome_tabela> ];
```

- **Exemplo:**

```sql
-- Adiciona uma nova coluna com um valor padrão.
ALTER TABLE usuarios ADD COLUMN ultima_atualizacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

```sql
-- Remove uma coluna existente.
ALTER TABLE usuarios DROP COLUMN celular;
```

```sql
-- Define um valor padrão para uma coluna existente.
ALTER TABLE usuarios ALTER COLUMN ativo SET DEFAULT TRUE;
```

```sql
-- Remove o valor padrão de uma coluna.
ALTER TABLE usuarios ALTER COLUMN ativo DROP DEFAULT;
```

```sql
-- Define uma coluna para não aceitar valores nulos.
ALTER TABLE usuarios ALTER COLUMN nome SET NOT NULL;
```

```sql
-- Permite que uma coluna aceite valores nulos.
ALTER TABLE usuarios ALTER COLUMN email DROP NOT NULL;
```

```sql
-- Altera o tipo de dados de uma coluna.
ALTER TABLE usuarios ALTER COLUMN nome TYPE VARCHAR(150);
```

```sql
-- Renomeia uma coluna.
ALTER TABLE usuarios RENAME COLUMN nome TO nome_completo;
```

```sql
-- Adiciona uma nova restrição de unicidade à coluna email.
ALTER TABLE usuarios ADD CONSTRAINT unique_email UNIQUE (email);
```

```sql
-- Remove uma restrição existente (ex: a restrição unique_email criada acima).
ALTER TABLE usuarios DROP CONSTRAINT unique_email;
```

```sql
-- Renomeia a tabela.
ALTER TABLE usuarios RENAME TO clientes;
```

TRUNCATE TABLE

Remove todos os registros de uma tabela, mas mantém sua estrutura de maneira que o comando TRUNCATE é consideravelmente mais rápido que o DELETE porque remove todos os registros de uma vez, sem a necessidade de varrer linha por linha. Essa operação gera menos logs de transação e, consequentemente, melhora o desempenho.

- **Sintaxe:**

```sql
TRUNCATE TABLE nome_da_tabela [ RESTART IDENTITY | CONTINUE IDENTITY ] [ CASCADE | RESTRICT ];
```

- **Exemplo:**

```sql
-- Remove todos os registros da tabela e reiniciar contador.
TRUNCATE TABLE usuarios RESTART IDENTITY;
```

DROP TABLE

Comando que gera a ação para apagar definitivamente uma tabela, eliminando-a por completo junto com todas as informações e estruturas de dados nela contidas.

- **Sintaxe:**

```sql
DROP TABLE [ IF EXISTS ] nome_da_tabela [ CASCADE | RESTRICT ];
```

- **Exemplo:**

```sql
-- Exclue a tabela referida.
DROP TABLE usuarios;
```

2.2. DML (Data Manipulation Language)

A Linguagem de Manipulação de Dados (DML) compreende os comandos essenciais utilizados para interagir e gerenciar os dados que estão armazenados nas tabelas de um

banco de dados. Esses comandos permitem aos usuários realizar operações como a inserção de novos registros (INSERT), a modificação de dados existentes (UPDATE), a exclusão de informações desnecessárias (DELETE) e a recuperação de dados específicos para visualização ou análise (SELECT).

INSERT INTO

Este comando é usado para adicionar novos registros (linhas, tuplas...) a uma tabela existente no banco de dados. Ele permite especificar os valores para cada coluna de forma individual.

- **Sintaxe:**

```
SQL
INSERT INTO <nome_da_tabela> (coluna1, coluna2, coluna3, ...)
VALUES (valor1, valor2, valor3, ...);
```

- **Exemplo**:

```
SQL
INSERT INTO usuarios (nome, email, contato_telefonico)
VALUES ('Maria Souza', 'maria.souza@example.com', '11987654321');
```

```
SQL
INSERT INTO usuarios (nome, email, contato_telefonico)
VALUES
  ('Ana Paula', 'ana.paula@example.com', '21912345678'),
  ('Pedro Santos', 'pedro.santos@example.com', '11998765432');
```

SELECT

Consulta e recupera dados de uma ou mais tabelas. É o comando mais utilizado em SQL para obter informações de um banco de dados.

- **Sintaxe:**

```
SQL
SELECT [DISTINCT] coluna1, coluna2, ...
FROM nome_da_tabela
[WHERE condição]
[GROUP BY coluna1, coluna2, ...]
[HAVING condição_de_grupo]
```

```
[ORDER BY coluna1 [ASC | DESC], coluna2 [ASC | DESC], ...]
|---|
```

- **Exemplo:**

```SQL
SELECT id_cliente, produto, valor
FROM pedidos
WHERE valor > 1000
ORDER BY valor DESC;
```

```SQL
SELECT DISTINCT produto
FROM pedidos
WHERE valor > 1000
ORDER BY valor DESC;;
```

UPDATE

Modifica registros existentes. A cláusula WHERE é crucial para evitar a atualização de todos os registros da tabela.

- **Sintaxe:**

```SQL
UPDATE nome_da_tabela
SET coluna1 = valor1, coluna2 = valor2, ...
[WHERE condição];
```

- **Exemplo:**

```SQL
UPDATE usuarios SET contato_telefonico = '21998877665'
WHERE email = 'maria.souza@example.com';
```

DELETE FROM

Remove registros de uma tabela. A cláusula WHERE é fundamental para especificar quais

registros remover.

- **Sintaxe:**

```
SQL
DELETE FROM nome_da_tabela [WHERE condição];
```

- **Exemplo:**

```
SQL
DELETE FROM usuarios WHERE id = 1;
```

2.3. DCL (Data Control Language)

Comandos para controlar o acesso aos dados. No PostgreSQL, as permissões são concedidas a roles (funções).

GRANT

Concede permissões a uma role/usuário.

- **Exemplo:**

```
SQL
CREATE ROLE analista LOGIN;
GRANT SELECT ON usuarios TO analista;
```

REVOKE

Revoga permissões previamente concedidas.

Exemplo:

```
SQL
REVOKE SELECT ON usuarios FROM analista;
```

2.4. TCL (Transaction Control Language)

Comandos que gerenciam transações, garantindo a atomicidade e consistência das operações.

BEGIN / COMMIT / ROLLBACK

Agrupa uma sequência de comandos em uma transação. COMMIT salva permanentemente as alterações, enquanto ROLLBACK as desfaz.

- **Exemplo:**

```SQL
BEGIN;
UPDATE usuarios SET ativo = FALSE WHERE id = 2;
COMMIT;
```

SAVEPOINT

Cria um ponto de salvamento dentro de uma transação, permitindo reverter parcialmente as operações para esse ponto.

- **Exemplo:**

```SQL
BEGIN;

UPDATE contas SET saldo = saldo - 100 WHERE id = 1;

SAVEPOINT antes_do_credito;

UPDATE contas SET saldo = saldo + 100 WHERE id = 99; -- Operação falha

ROLLBACK TO antes_do_credito; -- Desfaz apenas a segunda operação

UPDATE contas SET saldo = saldo + 100 WHERE id = 2; -- Operação correta

COMMIT;
```

3. Consultas Avançadas e Manipulação de Dados

O comando SELECT é a base para a recuperação de informações, permitindo especificar quais colunas e registros visualizar. Sua sintaxe pode ser enriquecida com diversas cláusulas para filtrar, ordenar e agrupar dados.

Cláusulas Principais

- **DISTINCT**: Elimina linhas duplicadas nos resultados.

```sql
SELECT DISTINCT cidade FROM clientes;
```

- **LIMIT**: Limita o número de linhas retornadas.

```sql
SELECT nome, idade FROM clientes LIMIT 5;
```

- **WHERE**: Filtra os registros com base em uma condição.

```sql
SELECT nome, idade FROM clientes WHERE idade > 30;
```

- **GROUP BY**: Agrupa os dados com base em uma ou mais colunas, geralmente usada com funções de agregação.

```sql
SELECT cidade, COUNT(*) AS total_clientes FROM clientes GROUP BY cidade;
```

- **ORDER BY**: Ordena o resultado da consulta.

```sql
SELECT nome FROM clientes ORDER BY nome ASC; -- Ascendente

SELECT nome FROM clientes ORDER BY nome DESC; -- Descendente
```

Funções de Agregação

Processam um conjunto de valores, geralmente de uma coluna, e retornam um único valor sumarizado. São frequentemente usadas com a cláusula GROUP BY.

- **COUNT(coluna | *):** Conta o número de linhas ou o número de valores não nulos em uma coluna especificada. COUNT(*), conta todas as linhas, COUNT(coluna), conta apenas as linhas onde a coluna não é nula.

```sql
SELECT COUNT(*) FROM usuarios; -- Total de usuários

SELECT COUNT(email) FROM usuarios; -- Total de emails não nulos
```

- **SUM(coluna numérica)**: Calcula a soma total dos valores em uma coluna numérica.

```sql
SELECT SUM(valor) FROM pedidos; -- Soma total de todos os pedidos
```

- **AVG(**coluna **numérica):** Calcula a média dos valores em uma coluna numérica.

```sql
SELECT AVG(salario) FROM funcionarios; -- Média salarial
```

- **MAX(coluna)**: Retorna o valor máximo em uma coluna. Pode ser usada com tipos numéricos, strings e datas.

```sql
SELECT MAX(data_pedido) FROM pedidos; -- Data do pedido mais recente

SELECT MAX(valor) FROM produtos; -- Produto de maior valor
```

- **MIN(**coluna**)**: Retorna o valor mínimo em uma coluna. Pode ser usada com tipos numéricos, strings e datas.

```sql
SELECT MIN(data_cadastro) FROM clientes; -- Data de cadastro mais antiga

SELECT MIN(preco) FROM produtos; -- Produto de menor preço
```

- **HAVING**: Filtra os grupos criados pelo GROUP BY.

```sql
SELECT cidade, COUNT(*) AS total_clientes
FROM clientes
GROUP BY cidade
HAVING COUNT(*) > 10;
```

- **STRING_AGG(expressão, delimitador)**: Concatena os valores de uma expressão em

uma única string, separados por um delimitador. Útil para listar itens relacionados em uma única linha.

```SQL
SELECT c.nome, STRING_AGG(p.produto, ', ') AS produtos_comprados
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.nome; -- Lista os produtos comprados por cada cliente
```

- **ARRAY_AGG(expressão)**: Agrega os valores de uma expressão em um array.

```SQL
SELECT categoria, ARRAY_AGG(nome_produto) AS produtos_da_categoria
FROM produtos
GROUP BY categoria; -- Cria um array de produtos por categoria
```

Ordem de Execução Lógica de uma Consulta SELECT

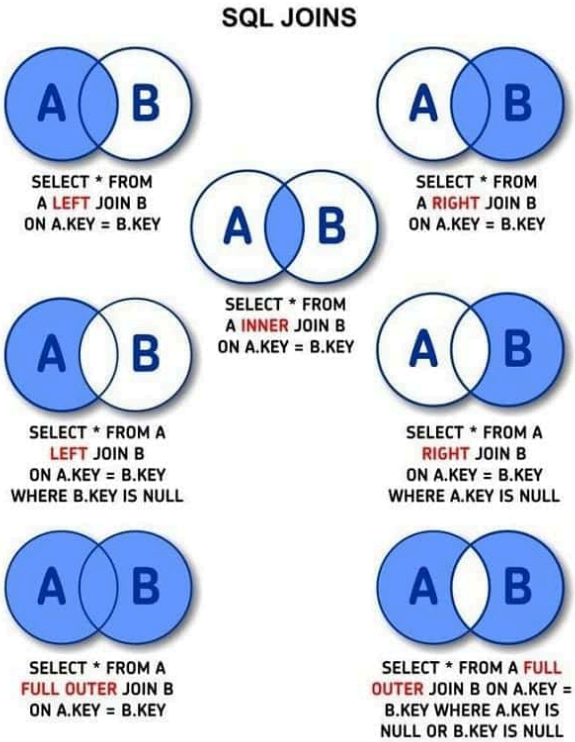
1. **FROM / JOIN**: Determina o conjunto de dados de trabalho.
2. **WHERE**: Filtra as linhas.
3. **GROUP BY**: Agrupa as linhas.
4. **HAVING**: Filtra os grupos.
5. **SELECT**: Seleciona as colunas.
6. **DISTINCT**: Remove duplicatas.
7. **ORDER BY**: Ordena o resultado final.
8. **LIMIT / OFFSET**: Pagina o resultado final.

Combinando Dados

Embora a Teoria dos Conjuntos seja um campo da matemática que lida com coleções de objetos, suas operações fundamentais têm uma analogia direta e prática no SQL. A maneira como combinamos e filtramos dados em bancos de dados relacionais é intrinsecamente ligada aos conceitos de união, interseção e diferença de conjuntos.

No contexto de SQL, cada resultado de uma consulta SELECT pode ser considerado um conjunto de tuplas (linhas). Os operadores de conjunto em SQL nos permitem manipular esses conjuntos de resultados de maneira poderosa.

Compreender a Teoria dos Conjuntos por trás dessas operações é crucial para escrever consultas SQL eficientes e logicamente corretas, permitindo a manipulação precisa dos dados.



INNER JOIN

Retorna apenas os registros onde a condição de junção é satisfeita em ambas as tabelas.

```sql
-- Lista apenas clientes que fizeram pedidos.
SELECT c.nome, p.produto, p.valor
FROM clientes c
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

LEFT JOIN (ou LEFT OUTER JOIN)

Retorna todos os registros da tabela da esquerda e os registros correspondentes da tabela da direita. Se não houver correspondência, as colunas da direita virão como NULL.

```sql
-- Lista todos os clientes e seus pedidos, se existirem.
SELECT c.nome, p.produto, p.valor
FROM clientes c
```

```
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

RIGHT JOIN (ou RIGHT OUTER JOIN)

Retorna todos os registros da tabela da direita e os registros correspondentes da tabela da esquerda. Se não houver correspondência, as colunas da esquerda virão como NULL.

```SQL
-- Lista todos os pedidos e os clientes que os fizeram, se existirem.
SELECT c.nome, p.produto, p.valor
FROM clientes c
RIGHT JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

FULL JOIN (ou FULL OUTER JOIN)

Retorna todos os registros de ambas as tabelas. Se não houver correspondência para um registro, as colunas da outra tabela virão como NULL.

```SQL
-- Lista todos os clientes e todos os pedidos, combinando-os onde for possível.
SELECT c.nome, p.produto, p.valor
FROM clientes c
FULL JOIN pedidos p ON c.id_cliente = p.id_cliente;
```

CROSS JOIN

Retorna o produto cartesiano: cada linha da primeira tabela é combinada com todas as linhas da segunda. Não requer uma condição ON.

```SQL
-- Combina cada cliente com cada pedido.
SELECT c.nome, p.produto
FROM clientes c
CROSS JOIN pedidos p;
```

SELF JOIN

Ocorre quando uma tabela é unida a si mesma. É útil para consultar dados hierárquicos, como um funcionário e seu gerente na mesma tabela.

```SQL
-- Lista cada funcionário e o nome do seu respectivo gerente.
SELECT
   f.nome AS funcionario,
   g.nome AS gerente
FROM
   funcionarios f
LEFT JOIN -- Usamos LEFT JOIN para incluir o CEO que não tem gerente
   funcionarios g ON f.id_gerente = g.id;
```

3.3. Subconsultas (Subqueries)

Uma subconsulta, também conhecida como consulta aninhada ou inner query, é uma instrução SELECT completa que é incorporada dentro de outra instrução SQL maior. Ela atua como um componente de uma consulta principal, fornecendo um conjunto de dados ou um valor escalar para ser usado pela consulta externa.

- **Subconsulta Escalar**: Retorna a um único valor.

```SQL
SELECT nome FROM funcionarios WHERE salario > (SELECT AVG(salario) FROM funcionarios);
```

- **Subconsulta de Múltiplas Linhas**: Retorna uma lista de valores, usada com operadores como IN, NOT IN, ANY, ALL.

```SQL
SELECT nome FROM produtos WHERE id_categoria IN (SELECT id FROM categorias WHERE nome =
'Eletrônicos');
```

- **Subconsulta Correlacionada**: A subconsulta interna depende da consulta externa para cada linha processada. Pode ser menos performática.

```SQL
SELECT f1.nome FROM funcionarios f1 WHERE f1.salario > (SELECT AVG(f2.salario) FROM funcionarios f2
WHERE f2.departamento_id = f1.departamento_id);
```

3.4. Common Table Expressions (CTEs)

Uma CTE, iniciada com a cláusula WITH, permite criar uma subconsulta nomeada e temporária, que existe apenas durante a execução da consulta. Melhora a legibilidade e a organização de consultas complexas.

Exemplo:

```SQL
WITH total_vendas AS (
    SELECT id_cliente, SUM(valor) AS total
    FROM pedidos
    GROUP BY id_cliente
)
SELECT c.nome, tv.total
FROM clientes c
JOIN total_vendas tv ON c.id_cliente = tv.id_cliente
WHERE tv.total > 1000;
```

- **CTE Recursiva**: Usada para consultar dados hierárquicos, como organogramas ou árvores de categorias.

  Imagine uma tabela de funcionários onde cada funcionário tem um id_gerente que aponta para seu superior. Queremos listar todos os funcionários e seu nível na hierarquia a partir do CEO.

```SQL
WITH RECURSIVE hierarquia_funcionarios AS (
    -- 1. Membro Âncora: Ponto de partida da recursão (o CEO, que não tem gerente).
    SELECT
        id,
        nome,
        id_gerente,
        0 AS nivel -- Nível 0 para o topo da hierarquia
    FROM
        funcionarios
    WHERE
        id_gerente IS NULL

    UNION ALL

    -- 2. Membro Recursivo: Une o resultado anterior (h) com a tabela original (f)
    -- para encontrar os subordinados diretos de cada nível.
    SELECT
        f.id,
        f.nome,
        f.id_gerente,
        h.nivel + 1 -- Incrementa o nível a cada passo da recursão
    FROM
```

```
    funcionarios f
  JOIN
    hierarquia_funcionarios h ON f.id_gerente = h.id
)
-- 3. Consulta final que lê os dados da CTE.
SELECT * FROM hierarquia_funcionarios ORDER BY nivel;
```

Subconsultas vs. CTEs: Diferenças e Aplicabilidade

Tanto subconsultas quanto CTEs permitem quebrar problemas complexos em etapas lógicas. No entanto, elas possuem diferenças importantes em legibilidade, reutilização e capacidades.

Característica	Subconsulta	CTE (Common Table Expression)
Legibilidade	Pode tornar a consulta principal complexa e difícil de ler, especialmente com múltiplos aninhamentos.	Melhora significativamente a legibilidade, separando a lógica em blocos nomeados e sequenciais.
Reutilização	Não é reutilizável. Precisa ser reescrita se a mesma lógica for necessária em outro ponto da consulta.	Pode ser referenciada múltiplas vezes na mesma consulta principal (após sua definição).
Recursividade	Não suporta consultas recursivas.	É a única forma de realizar consultas recursivas em SQL, essencial para dados hierárquicos.
Manutenção	Difícil de depurar e modificar, pois a lógica está aninhada dentro de outra instrução.	Mais fácil de manter e depurar, pois cada CTE pode ser testada de forma isolada.
Performance	O otimizador de consulta geralmente lida bem com subconsultas simples.	Para lógicas complexas e reutilizadas, o otimizador pode gerar um plano de execução mais eficiente.

Use uma Subconsulta quando:

1. A lógica é simples e não precisa ser reutilizada.
2. Precisa de um valor escalar rápido para uma cláusula WHERE ou SELECT.
3. A consulta é curta e o aninhamento não prejudica a legibilidade.

Use uma CTE quando:

1. A consulta é complexa e pode ser dividida em etapas lógicas.
2. Precisa realizar uma consulta recursiva para navegar em hierarquias.
3. A legibilidade e a manutenção do código são prioridades.

3.5. Operadores de Conjunto

Combinam os resultados de duas ou mais consultas SELECT.

- **UNION**: Combina os resultados e remove duplicatas existentes entre as tabelas.

```sql
SELECT coluna1 FROM tabela1
UNION
SELECT coluna1 FROM tabela2;
```

- **UNION ALL**: Combina resultados e mantém todas as linhas duplicadas.

```sql
SELECT coluna1 FROM tabela1
UNION ALL
SELECT coluna1 FROM tabela2;
```

- **INTERSECT**: Retorna apenas as linhas que aparecem em ambos os resultados.

```sql
SELECT coluna1 FROM tabela1
INTERSECT
SELECT coluna1 FROM tabela2;
```

- **EXCEPT**: Retorna as linhas da primeira consulta que não estão na segunda.

```sql
SELECT coluna1 FROM tabela1
EXCEPT
SELECT coluna1 FROM tabela2;
```

4. Estruturas Temporárias e Relacionamentos

As tabelas temporárias são visíveis apenas na sessão que foram criadas e são automaticamente descartadas no final da sessão.

Sintaxe:

```SQL
CREATE TEMPORARY TABLE temp_clientes (
    id_cliente INT,
    nome VARCHAR(100),
    cidade VARCHAR(50)
);
```

Uso Prático: São úteis para armazenar resultados intermediários em scripts complexos.

```SQL
-- Criando e populando a tabela temporária
CREATE TEMPORARY TABLE temp_vendas AS
SELECT id_cliente, SUM(valor_venda) AS total_vendas
FROM vendas
GROUP BY id_cliente;

-- Usando a tabela temporária em um JOIN
SELECT c.nome, v.total_vendas
FROM clientes c
JOIN temp_vendas v ON c.id_cliente = v.id_cliente;
```

4.2. Relacionamentos entre Tabelas

Relacionamentos são definidos usando chaves primárias (PRIMARY KEY) e estrangeiras (FOREIGN KEY) para garantir a integridade referencial dos dados.

```SQL
--Relacionamento Um-para-Muitos (1:N): Um cliente pode ter vários pedidos.
CREATE TABLE pedidos (
    id_pedido SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    data_pedido DATE NOT NULL,
    -- Definindo a chave estrangeira
    CONSTRAINT fk_cliente
```

```
    FOREIGN KEY (id_cliente) REFERENCES clientes (id_cliente)
    ON DELETE CASCADE
);
```

ON DELETE CASCADE: Se um cliente for excluído, seus pedidos também serão excluídos que estão relacionados.

ON UPDATE CASCADE: Se o id_cliente for atualizado na tabela clientes, a mudança será refletida na tabela pedidos.

Adicionando Relacionamentos com ALTER TABLE

É comum criar tabelas primeiro e definir os relacionamentos depois. O comando ALTER TABLE é usado para adicionar uma FOREIGN KEY a uma tabela já existente. Suponha que as tabelas clientes e pedidos foram criadas sem a chave estrangeira. Pode-se adicioná-la da seguinte forma:

```SQL
ALTER TABLE pedidos
ADD CONSTRAINT fk_pedidos_cliente
FOREIGN KEY (id_cliente) REFERENCES clientes (id_cliente)
ON DELETE CASCADE;
```

5. Otimização e Desempenho

Índices (Indexes)

Um índice é uma estrutura de dados que melhora a velocidade das operações de consulta, permitindo que o SGBD encontre dados sem percorrer a tabela inteira.

- **Vantagens**:
  - Acelera drasticamente as consultas (SELECT).
  - Melhora o desempenho de JOINs, ORDER BY e GROUP BY.
- **Desvantagens**:

Consome mais espaço de armazenamento.

Torna operações de escrita (INSERT, UPDATE, DELETE) mais lentas.

Índices em PostgreSQL

- **Índice Não-Clusterizado** (Non-Clustered): A estrutura do índice é separada dos dados da tabela. A ordem física dos dados na tabela não é alterada. Uma tabela pode ter múltiplos índices não-clusterizados.

```sql
-- Cria um índice na coluna de email para acelerar buscas por email.
CREATE INDEX idx_usuarios_email ON usuarios(email);
```

- **Índice "Clusterizado"**: O PostgreSQL não possui um índice clusterizado verdadeiro como o SQL Server, por exemplo. No entanto, o comando CLUSTER reorganiza fisicamente a tabela com base em um índice existente. Esta ordenação não é mantida automaticamente, o comando precisa ser executado novamente para reordenar a tabela após modificações.

```sql
-- Reordena fisicamente a tabela 'usuarios' com base no seu índice primário.
CLUSTER usuarios USING usuarios_pkey;
```

Planos de Execução no PostgreSQL

O plano de execução é uma representação detalhada de como o PostgreSQL pretende executar uma consulta. Ele é gerado pelo otimizador de consulta do SGBD e descreve a sequência de operações que serão realizadas para recuperar os dados, incluindo a ordem das junções, quais índices serão usados, e os métodos de acesso aos dados. Compreender o plano de execução é fundamental para otimizar o desempenho das consultas.

EXPLAIN
O comando EXPLAIN é a ferramenta principal para visualizar o plano de execução de uma consulta. Ele mostra as etapas que o otimizador escolheu.

- **Sintaxe:**

```sql
EXPLAIN <sua_consulta_sql>;
```

- **Exemplo:**

```sql
EXPLAIN SELECT * FROM produtos WHERE preco > 100 ORDER BY nome;
```

EXPLAIN ANALYZE

Além de mostrar o plano planejado, o EXPLAIN ANALYZE executa a consulta e adiciona informações de tempo de execução real e estatísticas. Isso permite comparar o plano estimado com o desempenho real e identificar gargalos.

- **Sintaxe:**

```sql
EXPLAIN ANALYZE <sua_consulta_sql>;
```

- **Exemplo:**

```sql
EXPLAIN ANALYZE SELECT c.nome, COUNT(p.id_pedido) FROM clientes c JOIN pedidos p ON c.id_cliente
= p.id_cliente GROUP BY c.nome;
```

Análise do Plano de Execução

Ao analisar um plano de execução, procure por:

Tipos de Operação:

- Seq Scan (Varredura Sequencial): A tabela inteira é lida linha por linha. É eficiente para tabelas pequenas ou quando uma grande porcentagem das linhas é qualificada. Para tabelas grandes, pode ser um sinal de que um índice está faltando ou não está sendo usado.
- Index Scan (Varredura de Índice): O PostgreSQL usa um índice para encontrar as linhas correspondentes. Geralmente é muito mais rápido que um Seq Scan para consultas seletivas.

- Index Only Scan: O PostgreSQL pode obter todos os dados necessários diretamente do índice, sem precisar acessar a tabela principal. É ainda mais eficiente que o Index Scan.
- Hash Join, Merge Join, Nested Loop Join: Diferentes algoritmos para combinar tabelas em operações JOIN. Cada um tem suas próprias características de desempenho dependendo do tamanho e ordenação dos dados.
- Sort: Indica que os dados estão sendo ordenados, o que pode ser custoso para grandes volumes de dados. ORDER BY ou GROUP BY podem resultar em um Sort.

Custos (Costs): Representam uma estimativa relativa do esforço de execução da consulta. Os custos são mostrados como (startup_cost..total_cost).

- startup_cost: Custo para retornar a primeira linha.
- total_cost: Custo para retornar todas as linhas.
- Quanto menores os custos, mais eficiente é o plano.

Linhas (Rows): Estimativa do número de linhas que a operação vai retornar. Comparar isso com o "actual rows" do EXPLAIN ANALYZE pode indicar problemas nas estatísticas do otimizador.

Largura (Width): Tamanho médio de cada linha em bytes.

Tempo de Execução (Actual Time - apenas com EXPLAIN ANALYZE): Mostra o tempo real levado para cada operação em milissegundos. Permite identificar exatamente onde a consulta está gastando mais tempo.

Dicas para Otimização Baseada em Planos de Execução:

- **Crie Índices Apropriados:** Se vir Seq Scan em tabelas grandes em cláusulas WHERE, JOIN ou ORDER BY, considere criar índices nas colunas relevantes.
- **Revise as Cláusulas WHERE:** Condições muito complexas ou funções em colunas indexadas podem impedir o uso de índices.
- **Ajuste os JOINs:** A ordem e o tipo de JOIN podem impactar drasticamente o desempenho.
- **Monitore as Estatísticas:** O PostgreSQL usa estatísticas sobre os dados para gerar planos. ANALYZE nas tabelas ajuda a manter essas estatísticas atualizadas.
- **Considere o uso de CTEs e Subconsultas:** Embora nem sempre melhorem o desempenho diretamente, podem tornar a consulta mais legível e, em alguns casos, permitir que o otimizador encontre um plano melhor.

MERGE (UPSERT)

O comando MERGE, também conhecido como UPSERT (de UPdate/inSERT), é uma declaração SQL condicional que permite combinar operações de INSERT, UPDATE e/ou DELETE em uma única instrução. Ele é extremamente útil para sincronizar dados entre uma tabela de origem (source) e uma tabela de destino (target), com base em uma condição de correspondência.

Disponibilidade: O comando MERGE não é um recurso nativo do PostgreSQL de forma direta, como é em outros bancos de dados (Oracle, SQL Server, MySQL 8+). No entanto, o PostgreSQL oferece alternativas que atingem o mesmo objetivo, como o INSERT ... ON CONFLICT (colunas) DO UPDATE (para UPSERT) ou combinações de CTEs e INSERT/UPDATE.

Para este guia, vamos focar na alternativa do PostgreSQL para o "UPSERT" que é o INSERT ... ON CONFLICT DO UPDATE, pois é o caso de uso mais comum do MERGE.

INSERT ... ON CONFLICT DO UPDATE (UPSERT no PostgreSQL)

Esta sintaxe é a maneira idiomática no PostgreSQL de realizar uma operação de UPSERT, ou seja, inserir uma linha se ela não existir, ou atualizar uma linha existente se houver um conflito em uma chave única ou primária.

Sintaxe:

```SQL
INSERT INTO <nome_da_tabela> (coluna1, coluna2, ...)
VALUES (valor1, valor2, ...)
ON CONFLICT (<coluna_de_conflito>) DO UPDATE
SET coluna1 = EXCLUDED.coluna1,
    coluna2 = EXCLUDED.coluna2,
    ...
WHERE <condicao_opcional>;
```

Explicação:

- ON CONFLICT (<coluna_de_conflito>): Especifica a coluna (ou conjunto de colunas) que possui uma restrição UNIQUE ou PRIMARY KEY. Se uma inserção tentar violar essa restrição, o conflito é detectado.
- DO UPDATE SET ...: Se um conflito for detectado, a linha existente será atualizada.
- EXCLUDED.<coluna>: Refere-se ao valor que *teria sido inserido* se não houvesse conflito. Isso permite que você use os novos valores da tentativa de inserção para atualizar a linha existente.
- WHERE <condicao_opcional>: Uma cláusula WHERE opcional pode ser adicionada ao

- DO UPDATE para aplicar a atualização apenas se certas condições forem atendidas.

Exemplo:

Imagine uma tabela com id_produto como chave primária. Precisa-se inserir novos produtos ou atualizar a quantidade e o preço se o produto já existir.

```SQL
-- Exemplo de criação da tabela
CREATE TABLE produtos (
    id_produto INT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    quantidade INT DEFAULT 0,
    preco DECIMAL(10, 2)
);

-- Inserindo ou atualizando produtos
INSERT INTO produtos (id_produto, nome, quantidade, preco)
VALUES
    (1, 'Laptop', 10, 1200.00),
    (2, 'Mouse', 50, 25.00),
    (1, 'Laptop', 15, 1150.00) -- Este causará um conflito e será atualizado
ON CONFLICT (id_produto) DO UPDATE
SET
    nome = EXCLUDED.nome,
    quantidade = produtos.quantidade + EXCLUDED.quantidade, -- Adiciona à
quantidade existente
    preco = EXCLUDED.preco;

-- Verificar o resultado
SELECT * FROM produtos;
```

No PostgreSQL, dentro de uma cláusula ON CONFLICT DO UPDATE do comando INSERT, a palavra-chave EXCLUDED é uma pseudo-tabela especial. Ela se refere à linha que a instrução INSERT tentou inserir, mas que causou um conflito (ou seja, a inserção falhou porque violaria uma restrição UNIQUE ou PRIMARY KEY).

Quando você usa ON CONFLICT (coluna_de_conflito) DO UPDATE SET coluna = EXCLUDED.coluna, você está dizendo: "Se houver um conflito na coluna_de_conflito, então, em vez de falhar a inserção, atualize a linha existente e use o valor que teria sido inserido (EXCLUDED.coluna) para a coluna em questão."

Isso é particularmente útil para operações de "UPSERT" (UPDATE or INSERT), onde você quer atualizar uma linha se ela já existe e inserir uma nova se não existe. EXCLUDED permite que você acesse os novos valores que você estava tentando introduzir.

Casos de Uso Comuns para UPSERT (MERGE):

1. **Sincronização de Dados:** Manter uma tabela atualizada com dados de uma fonte

externa, inserindo novos registros e atualizando os existentes.

2. **Carga de Dados Incrementais:** Ao carregar dados de um arquivo ou sistema externo, garantindo que as duplicatas sejam tratadas corretamente.
3. **Contadores e Agregações:** Atualizar contadores ou somas que podem ser incrementados ou criados.
4. **Tabelas de Log ou Status:** Registrar eventos ou estados, onde um evento para uma chave específica pode ser uma nova entrada ou uma atualização de uma entrada anterior.
5. **Gerenciamento de Cache:** Atualizar um cache de dados, onde você insere se o item não estiver no cache e atualiza se ele já estiver.

Para cenários mais complexos que envolvam operações de DELETE condicionais (além de INSERT e UPDATE), que o MERGE completo oferece, seria necessário no PostgreSQL usar combinações de CTEs, INSERT, UPDATE e DELETE separadamente, orquestradas em uma transação. No entanto, o INSERT ... ON CONFLICT DO UPDATE cobre a grande maioria dos casos de uso de UPSERT de forma elegante.

A detecção e tratamento de registros excluídos na origem, em um cenário de sincronização de dados (onde o MERGE completo seria ideal, mas não nativo no PostgreSQL), geralmente envolve estratégias que vão além de um simples UPSERT. O INSERT ... ON CONFLICT DO UPDATE do PostgreSQL lida apenas com inserções e atualizações. Para exclusões, você precisa de uma abordagem separada.

Existem algumas estratégias comuns para lidar com exclusões em cenários de sincronização:

1. Usando uma Coluna de Status/Flag (Soft Delete)
2. Usando um DELETE explícito com base na ausência (Diff)
3. Usando uma Tabela de Auditoria de Exclusões (Change Data Capture - CDC)